

II.2415 Advanced Algorithms and Programming

Lab 2: Revision of Basic Data Structures: Team T24

Members:

YADAV Anshuman Krishna (exercise 2, 4)

MAHALINGAM Nithees (exercise 1)

SARAVANAN Arun Prasath (exercise 3)

Github Link:

https://github.com/anshuman-krishna/advanced-algorithms-and-programming/tree/LAB_02_Revision

Exercise 1: MAHALINGAM Nithees

Friend Request Timeline

Input: A text message

Output:

- Number of uppercase letters
- Number of urgency punctuation marks (!, ?)
- Capital letter ratio
- Final message classification

Begin

Initialize uppercase_letters to 0

Initialize punctuation_marks to 0

Initialize total_letters to 0

Set previous_character to null

Set consecutive_count to 1

For each character in the message:

If the character is a letter:

Increase total_letters by 1

If the letter is uppercase:

Increase uppercase_letters by 1

If the character is '!' or '?':

Increase punctuation_marks by 1

If the character is the same as previous_character:

Increase consecutive_count by 1

Else:

Reset consecutive_count to 1

Update previous_character to the current character

II.2415 Advanced Algorithms and Programming

Lab 2: Revision of Basic Data Structures: Team T24

```
If total_letters equals 0:
Set caps_ratio to 0
Else:
Set caps_ratio to uppercase_letters divided by total_letters
If caps_ratio is at least 0.6 OR punctuation_marks is at least 5:
classification = "AGGRESSIVE"
Else if caps_ratio is at least 0.3 OR punctuation_marks is at least 3:
classification = "URGENT"
Else:
classification = "CALM"

Return all computed values
End
```

Complexity Analysis Questions:

1. How many character comparisons are needed for an n-character string?

Answer:

The algorithm checks the message once. Therefore, for each character, we perform a constant number of checks:

- Check if character is alphabetic -> hence, 1 comparison
- If alphabetic, check if uppercase -> 1 comparison
- Check if character is '!' or '?' -> will take up to 2 comparisons
- Check if character equals previous character -> it takes 1 comparison
- Possibly check if repeat count > 3 -> takes 1 comparison

So for each character, approx 5-7 comparisons are made.

Therefore, total comparisons : $c*n$

2. Can this be done in a single pass? What is the minimal number of passes required?

Yes.

All of the required information:

- Uppercase count
- Alphabetic count
- Punctuation count

II.2415 Advanced Algorithms and Programming

Lab 2: Revision of Basic Data Structures: Team T24

- Repeated character detection

is collected simultaneously during one traversal of the string.

Therefore, Minimal number of passes required : **1**

3. How would the complexity change if we support Unicode beyond ASCII?

Answer:

Time Complexity : **O(n)** (still)

Because:

- We still process each character once.
- Character classification (isalpha(), isupper()) remains constant-time per character.

Code Related Queries:

Reflection:

This algorithm is a single-pass analysis of message tone, which cuts down on computational costs and the need to scan the message multiple times. The constant space requirement also makes it ideal for longer messages.

The algorithm is rule-based, with caps ratio defining aggression, punctuation defining urgency, and repeated characters defining spam messages.

Strengths:

- Linear time complexity
- Constant space requirement
- Easily extendable

Weaknesses:

- Does not handle complex emotions well
- Does not actually analyze messages semantically
- Threshold values are heuristic-based

II.2415 Advanced Algorithms and Programming

Lab 2: Revision of Basic Data Structures: Team T24

Complexity Analysis

- **Time Complexity:** $O(N)$, we iterate through the N characters of the text message exactly once to update our counters.
- **Space Complexity:** $O(1)$, we only use a few basic variables for counting, which never grow regardless of the message length.

Test Cases:

Input	Output	Reason
""	CALM	Empty message
"hello"	CALM	No uppercase, no punctuation
"HELLO"	AGGRESSIVE	Caps ratio = 1.0
"Hey!!!"	URGENT	3 punctuations
"STOP!!!!!"	AGGRESSIVE	≥ 5 punctuations
"WHY ARE YOU DOING"	AGGRESSIVE	High Caps Ratio + punctuation
"ok?"	CALM	1 punctuation
"AaAaAa"	URGENT	Caps ratio = 0.5
"?????"	AGGRESSIVE	5 punctuation

Exercise 2: YADAV Anshuman Krishna

Mutual Friends Detection Using Sets

A. Intersection

```
FUNCTION Intersection(set1, set2) -> Set
result <- empty set
FOR EACH element IN set1 DO
IF set2.Contains(element) THEN
result.Add(element)
```

II.2415 Advanced Algorithms and Programming

Lab 2: Revision of Basic Data Structures: Team T24

```
END IF END FOR RETURN result
```

B. Difference

```
FUNCTION Difference(set1, set2) -> Set
result <- empty set
FOR EACH element IN set1 DO
IF NOT set2.Contains(element) THEN
result.Add(element)
END IF END FOR RETURN result
```

C. Union

```
FUNCTION Union(set1, set2) -> Set
result <- empty set
FOR EACH element IN set1 DO
result.Add(element)
END FOR
FOR EACH element IN set2 DO
IF NOT result.Contains(element) THEN
result.Add(element)
END IF END FOR RETURN result
```

D. Jaccard

```
FUNCTION JaccardSimilarity(set1, set2) -> Real
inter <- Intersection(set1, set2)
uni <- Union(set1, set2)

inter_size <- (inter)
uni_size <- (uni)
IF uni_size = 0 THEN
RETURN 0.0
END IF RETURN inter_size / uni_size
```

E. Friend Suggestion

```
FUNCTION SuggestFriends(user_friends, all_users_friends) -> Set
suggestions <- empty set
FOR EACH friend_id IN user_friends DO
friends_of_friend <- all_users_friends[friend_id]

FOR EACH candidate IN friends_of_friend DO
IF NOT user_friends.Contains(candidate) AND candidate != current_user_id THEN
suggestions.Add(candidate)
END IF END FOR
END FOR RETURN suggestions
```

Complexity Analysis Questions:

1. What is the time complexity of set intersection for sets of size m and n ?

Answer:

As we learnt in Lecture 4, the time complexity of set intersection for sets of size m and n would be:

- $O(\min(m,n))$ on average
- $O(m * n)$ for worst case scenario, and
- $O(1)$ for best case scenario

2. How does the representation (sorted array vs. hash set vs. bit array) affect complexity?

Answer:

Hash Set: Has fast lookups ($O(1)$) and Intersection is $O(m+n)$ or $O(\min(m,n))$.
Hence, making it good for sparse data.

Sorted Array: Lookups take $O(\log n)$ and Intersection takes $O(m+n)$ because we iterate through both arrays at the same time.

Bit Array: Has extremely fast bitwise operations (AND, OR).
Intersection is $O(U/W)$, where U is the total number of possible users and W is word_size. This generally is constant time relative to the number of friends but depends on the size.

3. For a network with millions of users, which representation would you choose and why?

Answer:

I would choose **Hash Set**, because, as mentioned above, it is good for sparse data, in a real social network the matrix is sparse, even with millions of users a normal person generally has friends ranging between 200 to 600.

If we compare it with other representations for the same case:

II.2415 Advanced Algorithms and Programming

Lab 2: Revision of Basic Data Structures: Team T24

- Bit Array: It would require a bit array of size million for every user, that will require higher computational values.
- Sorted Array: They are generally good for the scenario but inserting a new friend would be slow, around $O(N)$ due to shifting elements.
- Hash Set: Only stores the amount of friends, and adding a new friend is instant.

Code Related Queries:

Reflection:

By going through these set operations manually, it became clear why hash sets are better for social networks. The intersection function only needs to iterate through the first set because checking if an element exists in the second set happens instantly. This aligns well with the session objective to select optimal representations for concrete problems. If we had used standard arrays, finding mutual friends would require nested loops and drag the performance down heavily. It is a great practical lesson in picking the right data structure to save processing time.

Complexity Analysis

- **Intersection:**
 - Time: $O(m)$, where m is the size of the first set. We loop through it once and do constant-time lookups in the second set.
 - Space: $O(\min(m,n))$ to store the resulting mutual friends.
- **Difference:**
 - Time: $O(m)$ to iterate through the first set.
 - Space: $O(m)$ in the worst case where there is no overlap at all.
- **Union:**
 - Time: $O(m+n)$ where m and n are the sizes of the two sets. We process every element from both sets exactly once.
 - Space: $O(m+n)$ to hold the combined elements.
- **Jaccard Similarity:**
 - Time: $O(m+n)$, because it relies on computing both the intersection and the union.

II.2415 Advanced Algorithms and Programming

Lab 2: Revision of Basic Data Structures: Team T24

- Space: $O(m+n)$, to store the temporary intersection and union sets.
- **Suggest Friends:**
 - Time: $O(F*K)$, where F is the number of your friends and K is the average number of friends each of them has.
 - Space: $O(S)$ where S is the number of unique suggestions generated.

Test Cases

Input	Expected Output	Reason
Normal Case: • User A: {101, 102, 103, 104, 105} • User B: {103, 104, 106, 107, 108}	Mutual: {103, 104} Jaccard: 0.25	Tests standard operation. They share 2 friends out of 8 total unique friends, making the similarity score $2/8$ or 0.25.
Edge Case (No Mutual): • User A: {1, 2, 3} • User B: {4, 5, 6}	Jaccard: 0.0	Tests completely disjoint sets. The intersection is empty, so the similarity correctly drops to zero.
Edge Case (Identical): • User A: {1, 2} • User B: {1, 2}	Jaccard: 1.0	Tests perfect overlap. The intersection and union are exactly the same, resulting in a 100% match.
Edge Case (Empty Sets): • User A: set() • User B: set()	Jaccard: 0.0	Tests division by zero prevention. Since the union is zero, the algorithm safely catches it and returns 0.0 instead of crashing.
Friend Suggestions: Network: 1:{2,3}, 2:{1,4,5}, 3:{1,5,6}	Suggestions: {4, 5, 6}	Checks 2nd-degree connections. It successfully grabs the friends of User 2 and User 3, while filtering out User 1 (self) and direct friends.

II.2415 Advanced Algorithms and Programming

Lab 2: Revision of Basic Data Structures: Team T24

Exercise 3: SARA VANAN Arun Prasath

FriendRecommendation by Common Interests

Input: Users[U][I], FriendList, userID, K

Output: Top K similar users and recommended interests

```
BEGIN
CREATE empty list scores
FOR u <- 0 TO U-1 DO
IF u != userID AND u NOT IN FriendList[userID] THEN
dot <- 0
normA <- 0
normB <- 0

FOR i <- 0 TO I-1 DO
dot <- dot + Users[userID][i] * Users[u][i]
normA <- normA + Users[userID][i]^2
normB <- normB + Users[u][i]^2
END FOR

IF normA != 0 AND normB != 0 THEN
sim <- dot / (rootnormA * rootnormB)
ELSE
sim <- 0
END IF
ADD (u, sim) TO scores
END IF
END FOR
SORT scores in descending order
TopK <- first K users from scores
CREATE rec[I] <- 0

FOR EACH (u, sim) IN TopK DO
FOR i <- 0 TO I-1 DO
IF Users[userID][i] = 0 THEN
rec[i] <- rec[i] + Users[u][i] * sim
END IF
END FOR
```

II.2415 Advanced Algorithms and Programming

Lab 2: Revision of Basic Data Structures: Team T24

```
END FOR
RETURN TopK, rec
END
```

Complexity Analysis Questions:

1. What is the time complexity of computing all pairwise similarities for U users and I interests?

Answer:

To compute pairwise similarities:

- We compare every user with every other user -> this gives $U \times U = U^2$ comparisons.
- For each comparison, cosine similarity requires scanning all I interests -> $O(I)$.

Therefore, the total time complexity is $O(U^2 \times I)$

2. How would you optimize this for sparse matrices (most users have few interests)?

Answer:

If most values in the matrix are 0:

- Instead of storing the full matrix, store only non-zero interests using a hash map or sparse representation.
- Compute the dot product only on common interests.
- Use an inverted index (interest -> list of users) to avoid unnecessary comparisons.

This reduces computation significantly. Optimized time complexity becomes approximately $O(E)$. Where E is the number of non-zero entries.

3. What is the space complexity of storing the full matrix vs. sparse representations?

Answer:

- Full matrix stores all values -> $O(U * I)$
- Sparse representation stores only non-zero values -> $O(E)$

Where E is the number of non-zeros entities .

II.2415 Advanced Algorithms and Programming

Lab 2: Revision of Basic Data Structures: Team T24

Code Related Queries:

Reflection:

This codebase represents friend recommendation based on a user-interest matrix and cosine similarity. The algorithm compares the target user's interest vector to every other user and finds the Top-K most similar users who are not yet friends. Friend recommendations are then made using collaborative filtering by assigning weights to similar users' interests.

This method is quite efficient and straightforward, because:

- Cosine similarity measures similarity in interest patterns.
- Filtering avoids suggesting friends who are already friends.
- Weighted scoring enhances relevance in recommendations.
- The codebase is modular.

Strengths:

- The mathematical basis is well-defined (cosine similarity).
- Scalability is good for moderate numbers of users and interests, hence easy to modify and/or extend.
- Separates similarity computation and recommendation logic.

Weaknesses:

- Time complexity increases with the number of users ($O(U * I)$).
- Storing the entire matrix can be costly for large systems.
- The algorithm performs well when users have overlapping interests.
- The algorithm does not consider time or behavior patterns.

Complexity Analysis

- **Time Complexity:** $O(U * I + U \log U)$, we calculate similarity for U users across I interests ($U \times I$ steps), and then sort the resulting scores ($U \log U$ steps).
- **Space Complexity:** $O(U + I)$, we create a scores list to hold up to U users and a rec array to hold the I recommended interests.

II.2415 Advanced Algorithms and Programming

Lab 2: Revision of Basic Data Structures: Team T24

Test Cases:

Input	Output	Reason
U=3, I=6, User0 vs User1 (similar interests)	User1 in TopK	High cosine similarity due to overlapping interests
FriendList[0] = [1]	User1 NOT in TopK	Existing friends must be excluded
Users = [[0,0,0],[0,0,0]]	Similarity = 0	Both vectors have zero norm -> handled safely
U = 1	TopK = empty	No other users available for comparison
K > available users	Returns only available users	K cannot exceed valid candidates
Users = [[5,5,5],[5,5,5]]	Similarity = 1.0	Identical vectors -> maximum cosine similarity

Exercise 4: YADAV Anshuman Krishna

Mutual Followers Matrix

A. Base

```
STRUCTURE FollowerMatrix
matrix <- 2D Array of Boolean
size <- Integer
END STRUCTURE
```

```
FUNCTION InitializeGraph(n) -> FollowerMatrix
graph <- new FollowerMatrix
graph.size <- n
graph.matrix <- Create 2D Boolean Array of size n x n
RETURN graph
```

B. Follow and Unfollow

II.2415 Advanced Algorithms and Programming

Lab 2: Revision of Basic Data Structures: Team T24

```
PROCEDURE Follow(graph, follower, followee)
graph.matrix[follower][followee] <- TRUE
END PROCEDURE
```

```
PROCEDURE Unfollow(graph, follower, followee)
graph.matrix[follower][followee] <- FALSE
END PROCEDURE
```

```
FUNCTION Is_following(graph, follower, followee) -> Boolean
RETURN graph.matrix[follower][followee]
```

C. Following Unfollowing List

```
FUNCTION Get_followers(graph, user) -> Array
followers_list <- empty list
n <- graph.size
FOR i <- 1 TO n DO
IF graph.matrix[i][user] = TRUE THEN
followers_list.Add(i)
END IF END FOR
RETURN followers_list
```

```
FUNCTION Get_following(graph, user) -> Array
following_list <- empty list
n <- graph.size
FOR i <- 1 TO n DO
IF graph.matrix[user][i] = TRUE THEN
following_list.Add(i)
END IF END FOR
RETURN following_list
```

D. Mutuals

```
FUNCTION FindMutualFollows(graph) -> list of pairs
mutuals <- empty list
n <- graph.size
FOR i <- 1 TO n DO
FOR j <- 1 TO n DO
is_following <- graph.matrix[i][j]
is_followed_back <- graph.matrix[j][i]
IF i != j AND is_following AND is_followed_back THEN
mutuals.Add(pair(i, j))
END IF END FOR
END FOR RETURN mutuals
```

E. Influence

```
FUNCTION CalculateInfluence(graph, user) -> Real
n <- graph.size
followers <- Length(Get_followers(graph, user))
following <- Length(Get_following(graph, user))
total_score <- followers + following
result <- total_score / n
RETURN result
```

Complexity Analysis Questions:

1. What is the time complexity of Get_followers() and Get_following()?

Answer:

O(N) (Linear Time), because, even though accessing a single cell is instant, i.e. $O(1)$, to retrieve a list of followers, we are forced to scan the entire row or column of length N to find true values, without skipping any cells.

2. What is the space complexity of the adjacency matrix for N users?

Answer:

O(N²), because, an adjacency matrix is a square grid, if we have a thousand users we will need a grid of $1000 * 1000 = 10^6$ cells, hence, the required space grows quadratically.

3. At what network size does this become impractical? Why?

Answer:

It becomes impractical at roughly 50,000 to 100,000 users.

Reason 1: Memory ($O(n^2)$). The matrix is a square grid, at 100,000, the grid would be $100k * 100k$, i.e. 10 billion cells, normally, in programming languages, this single grid would take up to 10 GBs of RAM.

Reason 2: The real problem is that we are not utilising all of it since people only follow a tiny fraction of the total user-base, so, out of these 10 billion cells, 9 to 9.99 billion cells are just storing “false”, hence practically burning massive amounts of expensive server memory just to hold zeros.

II.2415 Advanced Algorithms and Programming

Lab 2: Revision of Basic Data Structures: Team T24

Code Related Queries:

Reflection

Implementing the adjacency matrix clearly demonstrated the pros and cons of this data structure. On one hand, checking if a specific user follows another is incredibly fast and only requires a single constant-time edge lookup. On the other hand, finding a user's total followers forces us to iterate through the entire user-base, making it less efficient for that specific operation. Plus, seeing how many “False” values pile up makes sense as to why giant networks cannot use this method. It just wastes too much server memory on empty connections.

Complexity Analysis

N: total number of users in the network

- **Initialize Graph:**
 - Time: $O(N^2)$, to create and fill every cell in the grid with “False”.
 - Space: $O(N^2)$, to store the 2D array in memory (massive).
- **Follow / Unfollow / IsFollowing:**
 - Time: $O(1)$, we directly access a specific row and column instantly.
 - Space: $O(1)$, no extra memory is created for these checks.
- **GetFollowers / GetFollowing:**
 - Time: $O(N)$, to scan an entire row or an entire column to find the "True" values.
 - Space: $O(K)$, where K is the actual number of followers or people the user is following.
- **Find-Mutual-Follows:**
 - Time: $O(N^2)$, to scan every single cell in the entire matrix to check for bidirectional relationships.
 - Space: $O(M)$, where M is the number of mutual follower pairs we find and store in the list.
- **Calculate-Influence:**
 - Time: $O(N)$, because it relies on the Get_followers and Get_following functions.

II.2415 Advanced Algorithms and Programming

Lab 2: Revision of Basic Data Structures: Team T24

- Space: $O(K)$, to temporarily hold the lists generated by those functions before counting their length.

Test Cases

Input	Expected Output	Reason
Normal Case: 3 Users. User 0 follows 1. User 1 follows 0 and 2.	• Followers of User 1: [0] • User 1 follows: [0, 2] • Mutuals: [(0, 1), (1, 0)] • Influence of 1: 1.0	Verifies basic matrix logic. User 1 has 1 follower and follows 2 people. $(1 + 2) / 3$ users gives an influence score of 1.0.
Edge Case (Self-Following): User 2 follows User 2.	Mutuals: Does not include (2, 2)	Ensures the algorithm does not count a user following themselves as a "mutual friendship". The $i \neq j$ logic successfully filters this out.
Edge Case (Unfollowing): User 1 unfollows User 0.	Mutuals: [] (Empty list)	Verifies that breaking a bidirectional link updates the system correctly. Since the following is now only one-way, the mutual pair is removed.

the end. merci.